

Automated training pipeline (Part 2): progress

Automated training pipeline (Part 2): progress

My hands-on log for Part 2: the config-driven training pipeline that takes the Part 1 dataset, trains an adapter, and gates it on two axes. Research plus experiment, not a final build. Runs on top of ../task dataset generation pipeline1/. Train on Kaggle/Colab; keep small.

Status: [] not started, [~] in progress, [x] done

Stage / experiment	What	Status
Configure	rules propose config, human confirms (config = single source of truth)	[~]
Train	config-driven LoRA run	[~]
Guardrails	replay mix (+ documented LR/regularization/merge/early-stop knobs)	[~]
Evaluate	two-axis (task gold + sentinel/reasoning/tools)	[~]
Gate	accept iff task gain >= min AND regression <= max; one adjusted re-run	[~]
Exp 1: map landscape	research Axolotl / LLaMA-Factory / torchtune / TRL / Unsloth	[]
Exp 2: small config run	pipeline.py on the Part 1 dataset	[]
Exp 3: exercise guardrails	no-replay vs replay, compare regression axis	[]
Exp 4: run the gate	both axes vs acceptance criteria, failed gate -> adjusted re-run	[]
Write-up	WRITEUP.md filled with my numbers	[]

(The pipeline CODE is complete and offline-validated; the training/eval RUNS are pending, which is what keeps the rows at [~] and the experiments at [].)

What is built

- ☒ config.yaml: the single source of truth (model, LoRA, data mix, hyperparams, guardrails, acceptance thresholds, adjust-on-fail, confirm gate).
- ☒ propose_config.py: rules propose a config draft from the dataset (-> config.proposed.yaml).

- ☒ train_from_config.py: config-driven LoRA/QLoRA, replay vs no-replay variant.
- ☒ evaluate_from_config.py: two-axis eval, importable + CLI.
- ☒ gate.py: acceptance gate (task gain + worst regression drop), pure logic.
- ☒ pipeline.py: orchestrator (base -> train -> eval -> gate -> one adjusted re-run) and the guardrail experiment mode.

To run (pending, do these manually)

- ☐ Setup: pyyaml present; venv ready; confirm config.yaml (confirmed: true).
- ☐ propose_config.py , review, fold into config.yaml.
- ☐ pipeline.py (gate loop) -> read runs/report.json and the verdict.
- ☐ pipeline.py --experiment guardrails -> read the no-replay vs replay regression numbers.
- ☐ Record base vs fine-tuned for the task + the gate verdict in WRITEUP.md.

Research (Exp 1) notes

Results to fill in (Exp 2-4)

- base vs candidate (task macro-F1, sentinel, reasoning, tools):
- guardrail effect (no-replay vs replay on the regression axis):
- gate verdict + whether the adjusted re-run was needed:

What surprised me / open questions
